

Resumo de Estudos — Prova II de Inteligência Artificial

Baseado no “Guia para estudos para Prova II de 2026”. Cobre os 6 blocos do guia: Algoritmos Genéticos, Redes Neurais Artificiais, SVM, Rede Bayesiana/Árvores de Decisão, Lógica Fuzzy e Sistemas Especialistas.

1. Algoritmos Genéticos (AGs)

Resumo: técnica de otimização/busca inspirada na seleção natural. Uma população de soluções (cromossomos) evolui por gerações via seleção, cruzamento e mutação, guiada por uma função de aptidão (fitness).

1.1 Funcionamento geral

- **Cromossomo:** codificação de uma solução candidata (geralmente um vetor binário ou de números reais).
- **População:** conjunto de cromossomos avaliados a cada geração.
- **Ciclo do AG:** avaliar aptidão → selecionar pais → cruzar → mutar → substituir população → repetir até critério de parada (nº de gerações ou aptidão-alvo).

1.2 Cálculo de aptidão (fitness)

- Função de erro/qualidade que mede quão boa é cada solução para o problema.
- Quanto maior a aptidão, maior a chance de o indivíduo ser selecionado para reprodução.

1.3 Seleção por roleta (roulette wheel)

- Cada indivíduo ocupa uma “fatia” da roleta proporcional à sua aptidão relativa: $P(i) = \text{aptidão}(i) / \sum \text{aptidão}(\text{todos})$.
- Sorteia-se um número aleatório e verifica-se em qual fatia ele cai — indivíduos mais aptos têm mais chance de serem escolhidos, mas os menos aptos não são excluídos por completo (mantém diversidade).

1.4 Operadores genéticos

- **Cruzamento (crossover):** combina genes de dois pais para gerar filhos (ex.: corte em um ponto e troca de segmentos). Taxa típica: 80–90%. Quanto maior a taxa, mais rápido novas combinações surgem na população.
- **Mutação:** altera aleatoriamente um gene do cromossomo (ex.: inverte um bit). Taxa típica baixa: 10–30% (ou muito menor, a depender do problema). Evita estagnação em ótimos locais e mantém diversidade genética.

1.5 AGs aplicados à evolução de RNAs (neuroevolução)

- Em vez de treinar pesos por backpropagation, o AG evolui diretamente os pesos (e às vezes a topologia) da rede.
- Cada indivíduo da população = um conjunto de pesos (ou uma arquitetura) de RNA; a aptidão é o desempenho da rede na tarefa.
- **NEAT** (Neuroevolution of Augmenting Topologies) é o exemplo clássico: evolui pesos e topologia simultaneamente, começando de redes simples e complexificando gradualmente; mutações incluem perturbar pesos, adicionar conexões e adicionar neurônios.
- **Vantagem:** não depende de gradiente (útil quando a função de perda não é diferenciável, ex.: aprendizado por reforço). **Desvantagem:** geralmente converge mais devagar que back-propagation em problemas supervisionados clássicos.

2. Redes Neurais Artificiais (RNAs)

2.1 Perceptron

- Classificador linear binário mais simples, proposto por Rosenblatt (1958).
- Recebe entradas $x_1 \dots x_n$, aplica pesos $w_1 \dots w_n$ e um bias, soma tudo ($s = \sum w_i \cdot x_i + \text{bias}$) e passa por uma **função de ativação degrau**: se $s \geq 0 \rightarrow$ ativa (saída 1); senão $\rightarrow$ desativa (saída 0/-1).
- Só resolve problemas **linearmente separáveis** (limitação clássica: não resolve XOR).

2.2 Multilayer Perceptron (MLP)

- Rede com uma ou mais **camadas ocultas** entre entrada e saída, cada neurônio conectado a todos da camada seguinte (fully connected).
- Usa funções de ativação não lineares (ex.: sigmoide, ReLU), o que permite aprender fronteiras de decisão não lineares.
- Pelo **teorema de aproximação universal** (Cybenko), um MLP com uma camada oculta e ativação sigmoide pode aproximar qualquer função contínua, dado neurônios suficientes.

2.3 Função de ativação

- Introduce não linearidade; sem ela, uma rede de várias camadas equivaleria a uma única transformação linear.
- Aplicada sobre a soma ponderada das entradas de cada neurônio, define a saída dele.

2.4 Cálculo de classificação (forward pass)

Para cada neurônio: $s = \sum (w_i \cdot x_i) + \text{bias} \rightarrow y = f(s)$, onde f é a função de ativação. Isso se repete camada a camada até a saída. Exemplo simples (1 neurônio, ativação degrau):

Entradas: $x_1=1$, $x_2=0$

Pesos: $w_1=0.8$, $w_2=-0.5$, $\text{bias}=-0.1$

$s = (1 \times 0.8) + (0 \times -0.5) + (-0.1) = 0.7$

$y = \text{degrau}(0.7) = 1$ (pois $s \geq 0$)

2.5 Como o treinamento afeta os pesos

- Os pesos começam aleatórios; a cada exemplo de treino, a rede compara sua saída com o valor esperado e ajusta os pesos para reduzir o erro.
- Exemplos de treinamento “moldam” a fronteira de decisão: dados enviesados ou pouco representativos geram pesos enviesados (a rede aprende exatamente os padrões presentes nos dados, nada além disso).

2.6 Função de perda (loss function)

- Mede a diferença entre a saída prevista e a saída desejada (ex.: erro quadrático médio, entropia cruzada).
- É o valor que o treinamento tenta minimizar.

2.7 Atualização de pesos (regra delta / backpropagation)

- Fórmula conceitual: $\text{peso_novo} = \text{peso_atual} - \text{taxa_de_aprendizado} \times \text{gradiente_do_erro}$.
- No formato do slide: $W_{11} = W_{11} + \eta \cdot \Delta W_{11}$, onde η (eta) é a taxa de aprendizado e ΔW_{11} é a correção sugerida pelo erro propagado (regra delta) para aquela conexão específica.

- **Backpropagation:** propaga o erro da saída para trás, camada a camada, usando a regra da cadeia para calcular o gradiente de cada peso.
- Taxa de aprendizado (η): se muito alta, o treino “pula” o mínimo e não converge; se muito baixa, o treino fica lento.

2.8 Gradiente descendente (conceito)

- Método de otimização que ajusta os pesos na direção **oposta ao gradiente** da função de perda (direção de maior crescimento do erro), para caminhar em direção ao mínimo do erro.
- Backpropagation calcula os gradientes; gradiente descendente usa esses gradientes para de fato atualizar os pesos.

2.9 Overfitting e Underfitting

	Overfitting	Underfitting
Definição	Modelo “decora” os dados de treino	Modelo não aprende nem o padrão básico
Sintoma	Alto desempenho no treino, baixo em teste/validação	Baixo desempenho em treino e teste
Causa comum	Modelo complexo demais, poucos dados, treino excessivo	Modelo simples demais, treino insuficiente
Solução	Regularização, dropout, mais dados, early stopping	Aumentar complexidade do modelo, treinar mais

2.10 Épocas

- Uma época = uma passada completa da rede por **todo** o conjunto de treinamento (todos os exemplos, uma vez).

2.11 Bias / Viés (dois tipos)

- **Bias como parâmetro do neurônio:** um peso adicional (não ligado a nenhuma entrada) que desloca a função de ativação, dando à rede mais flexibilidade para ajustar a fronteira de decisão.
- **Bias como viés estatístico/erro sistemático** (trade-off bias-variância): erro por suposições simplificadoras do modelo. Alto bias → underfitting; alta variância → overfitting.

2.12 Batches (lotes)

- Em vez de atualizar os pesos após cada exemplo (ou após todo o dataset), divide-se o treino em lotes (batches) de N exemplos; os pesos são atualizados após cada lote.
- Batch size afeta velocidade de treino, uso de memória e estabilidade da convergência.

2.13 Técnicas de regularização

- **Objetivo:** reduzir overfitting, penalizando a complexidade do modelo para melhorar a generalização.
- **Principais técnicas vistas em aula:**
 - **L1 (Lasso):** soma dos valores absolutos dos pesos como penalidade; tende a zerar pesos pouco importantes (gera esparsidade / seleção de variáveis).
 - **L2 (Ridge):** soma dos quadrados dos pesos como penalidade; encolhe os pesos, mas raramente os zera.

- **Dropout**: desliga aleatoriamente neurônios durante o treino, forçando a rede a não depender demais de neurônios específicos.
- **Early stopping**: interrompe o treino quando o erro de validação para de melhorar (mesmo que o erro de treino ainda caia).
- Não é preciso saber a fórmula exata de L1/L2, só entender que ambas somam uma penalidade proporcional à magnitude dos pesos à função de perda.

2.14 CNNs (Redes Neurais Convolucionais)

- **Arquitetura geral**: camada convolucional → camada de pooling → (repete) → camada(s) totalmente conectada(s) → saída.
- **Etapas**:
 - **Convolução**: filtros (kernels) percorrem a imagem detectando padrões locais (bordas, texturas, formas), gerando mapas de características.
 - **Pooling**: reduz a dimensão dos mapas de características (ex.: max pooling pega o maior valor de uma janela 2×2), resumindo informação e reduzindo custo computacional.
 - **Camada totalmente conectada**: interpreta os mapas de características extraídos e produz a classificação final.
- **Vantagens**: preserva a organização espacial dos pixels; muito menos pesos que uma rede densa equivalente (compartilhamento de filtros); ótima para dados em grade (imagens).
- **Desvantagens**: precisa de muitos dados rotulados; custo computacional alto para treinar; menos intuitiva para dados não espaciais.
- **Aplicações**: classificação de imagens, detecção/reconhecimento de objetos, reconhecimento facial, segmentação semântica.

2.15 RNNs (Redes Neurais Recorrentes)

- **Arquitetura geral**: possuem conexões recorrentes — a saída de um passo de tempo realimenta a entrada do próximo, criando uma “memória” de curto prazo. Variante mais usada: **LSTM**, com portões (gates) que controlam o que é lembrado/esquecido, mitigando o problema de dependências longas.
- **Vantagens**: naturalmente adequadas a dados sequenciais/temporais; LSTM lida bem com dependências de longo prazo.
- **Desvantagens**: treino sequencial (difícil de paralelizar), mais lento; LSTM é mais complexa e custosa computacionalmente que RNN simples; perde força frente a Transformers em sequências muito longas.
- **Aplicações**: reconhecimento de fala, tradução automática, geração de texto, séries temporais, análise de sentimento.

2.16 LLMs — tokens, Softmax e exemplo

- **Token**: unidade mínima de texto processada pelo modelo — pode ser uma palavra inteira ou um pedaço dela (ex.: “sanduíche” → “sand” + “uíche”); palavras comuns costumam virar um único token.
- **Ideia central**: um LLM prevê o próximo token dada uma sequência, calculando uma probabilidade para cada token possível do vocabulário.
- **Pipeline simplificado**: texto → tokens → embeddings (vetores) → rede neural (Transformer) → camada linear (um valor/logit por token do vocabulário) → **Softmax** (transforma os logits em probabilidades que somam 1) → escolhe/amostra o próximo token.
- **Exemplo conceitual** (frase pequena, tipo material de aula): dado “Eu vi um...”, a rede calcula um score para cada palavra candidata do vocabulário (“gato”, “carro”, “livro”...) e o Softmax converte esses scores em probabilidades (ex.: gato 60%, carro 25%, livro 15%) — escolhe-se a mais provável (ou amostra-se conforme a distribuição).

2.17 Transformers

- **Etapas principais:** embedding das palavras/tokens → codificação posicional (positional encoding, pois o Transformer não processa em ordem sequencial como a RNN) → camadas de self-attention (+ feed-forward) → saída.
- **Self-attention / matriz de atenção:** cada token gera três vetores — **Query (Q)**, **Key (K)**, **Value (V)** — via multiplicação por matrizes de peso aprendidas. A relevância entre tokens é o produto escalar $Q \cdot K$; esses scores passam por **Softmax** (normalizam para somar 1) e ponderam os vetores V, gerando uma representação de cada token que leva em conta o contexto da frase inteira.
- **Multi-head attention:** repete esse processo várias vezes em paralelo (com diferentes projeções), capturando diferentes tipos de relação entre os tokens.
- **Aplicações:** LLMs (GPT, BERT), tradução automática, Vision Transformers (imagens), geração de texto/código.
- **Vantagem sobre RNN:** processa a sequência inteira em paralelo (mais rápido de treinar) e captura dependências de longo alcance melhor.

2.18 Gráficos das funções de ativação e uso recomendado

- **Degrau:** saída binária (0 ou 1); usada no perceptron clássico (histórico, pouco usada hoje).
- **Sigmoide:** forma em “S”, saída entre 0 e 1; boa para saída de classificação binária; sofre com gradiente que desaparece (vanishing gradient) em redes profundas.
- **Tanh:** forma em “S” simétrica, saída entre -1 e 1; centrada em zero (geralmente melhor que sigmoide em camadas ocultas), mas mesmo problema de gradiente que desaparece em redes muito profundas.
- **ReLU:** zero para valores negativos, linear (identidade) para positivos; função mais usada em **camadas ocultas de CNNs e MLPs modernas** por ser barata computacionalmente e reduzir o problema do gradiente que desaparece.
- **Softmax:** converte um vetor de valores em uma distribuição de probabilidade (soma 1); usada na **camada de saída** de problemas de classificação multiclasse (ex.: LLMs, classificadores de várias categorias).
- **Regra prática:** ReLU nas camadas ocultas (CNN/MLP); sigmoide na saída binária; Softmax na saída multiclasse; tanh/variações de gate (sigmoide/tanh) nas RNNs/LSTM.

3. SVM (Support Vector Machine)

Resumo: classificador supervisionado que busca o hiperplano que separa as classes com a **maior margem possível**.

- **Hiperplano:** fronteira de decisão definida por $g(x) = w^T x + w_0 = 0$. Pontos com $g(x) > 0$ são de uma classe, $g(x) < 0$ da outra.
- **Vetores de suporte:** os pontos de dado mais próximos do hiperplano; são eles que efetivamente “sustentam”/definem a posição da fronteira (os demais pontos poderiam ser removidos sem alterar o resultado).
- **Margem:** distância entre o hiperplano e os vetores de suporte mais próximos de cada classe. O SVM escolhe o hiperplano que **maximiza essa margem** (classificador de margem máxima).
- **Classificação de um ponto novo:** calcula-se $g(x) = w^T x + w_0$; o sinal do resultado indica a classe.
- **Pelo gráfico:** o hiperplano é a linha (2D) ou plano (3D+) que separa visualmente os grupos de pontos, equidistante das margens de cada lado.
- **Observação:** para dados não linearmente separáveis, usa-se o “kernel trick” (mapeia os dados para uma dimensão maior onde a separação linear é possível) — vale saber que existe, mesmo que o cálculo detalhado não seja cobrado.

4. Rede Bayesiana e Árvores de Decisão (conceitual, sem cálculo)

4.1 Rede Bayesiana

- Modelo probabilístico representado por um **grafo direcionado acíclico**: nós = variáveis; arestas = dependências condicionais entre elas.
- Cada nó tem uma tabela de probabilidade condicional, dado o estado de seus “pais” no grafo.
- Usada para raciocínio sob incerteza e inferência a partir de dados incompletos (ex.: diagnóstico médico, dado sintoma observado, qual a probabilidade de cada doença).
- **Vantagem**: lida naturalmente com incerteza e permite inferência em ambas as direções (causa→efeito e efeito→causa).

4.2 Árvores de Decisão

- Estrutura hierárquica (em árvore) em que cada nó interno representa um teste sobre um atributo, cada ramo um resultado do teste, e cada folha uma classe/decisão final.
- Construída dividindo os dados recursivamente pelo atributo que melhor separa as classes (na prática, calculado por métricas como entropia/ganho de informação — não cobrado o cálculo).
- **Vantagem**: fácil de interpretar e visualizar (“a mais fácil de explicar”, pois a maioria das pessoas já entende árvores hierárquicas de decisão).
- **Desvantagem**: tende a overfitting se crescer demais (é comum limitar a profundidade ou podar a árvore).

5. Lógica Fuzzy (operadores básicos)

Resumo: lógica que permite graus de pertinência entre 0 e 1 (não apenas verdadeiro/falso), útil para representar conceitos imprecisos (“quente”, “alto”, “rápido”).

- **Grau de pertinência $\mu(x)$** : o quanto um elemento pertence a um conjunto fuzzy, variando continuamente entre 0 (não pertence) e 1 (pertence totalmente) — diferente da lógica clássica, onde só existe 0 ou 1.
- **NÃO (NOT/complemento)**: $\mu(\text{NÃO } A) = 1 - \mu(A)$.
- **E (AND/interseção)**: $\mu(A \text{ E } B) = \min(\mu(A), \mu(B))$.
- **OU (OR/união)**: $\mu(A \text{ OU } B) = \max(\mu(A), \mu(B))$.

Exemplo prático (variável linguística “Altura”): Alice é alta com $\mu=55\%$, Bob com $\mu=75\%$, Carlos com $\mu=100\%$. - Carlos NÃO é alto $\rightarrow 1 - 1.00 = 0\%$. - Bob NÃO é alto $\rightarrow 1 - 0.75 = 25\%$. - Alice E Bob são altos $\rightarrow \min(0.55, 0.75) = 55\%$. - Alice OU Bob são altos $\rightarrow \max(0.55, 0.75) = 75\%$.

6. Sistemas Especialistas (conceitos gerais)

- Software que imita o processo de raciocínio de um especialista humano em um domínio específico (ex.: diagnóstico médico, suporte técnico).
- **Três componentes principais**:
 - **Base de conhecimento**: fatos e regras (geralmente do tipo “SE... ENTÃO...”) que representam o conhecimento do especialista sobre o domínio.
 - **Motor de inferência**: aplica as regras da base de conhecimento aos fatos disponíveis para deduzir novas conclusões (verifica quais regras “casam” com os fatos conhecidos e as dispara).

- **Interface com o usuário:** permite que o usuário forneça fatos/perguntas e receba as conclusões/recomendações do sistema.
 - **Vantagem:** captura e preserva conhecimento especializado, decisões consistentes e explicáveis (a cadeia de regras usada pode ser mostrada).
 - **Limitação:** não aprende sozinho com novos dados (ao contrário de RNAs) — precisa que um especialista/engenheiro do conhecimento atualize manualmente a base de regras.
-

Fontes consultadas

- [Algoritmos Genéticos — USP](#)
- [Algoritmos Genéticos — UFSC](#)
- [Algoritmos Genéticos — Um Resumo Detalhado](#)
- [Neuroevolução com NEAT](#)
- [Treinamento de Redes Neurais com Algoritmos Genéticos](#)
- [Perceptron e Multilayer Perceptron](#)
- [Perceptron multicamadas — Wikipédia](#)
- [Neurônios Artificiais - Função de Ativação](#)
- [Backpropagation Parte 2 — Deep Learning Book](#)
- [Gradiente Descendente — Data Science Academy](#)
- [Overfitting e Underfitting — NeuroDataAI](#)
- [Regularização L1/L2 — IA Com Café](#)
- [Redes Neurais Convolucionais — Data Hackers](#)
- [Camadas de Pooling — Deep Learning Book](#)
- [LSTM — Deep Learning Book](#)
- [Tokens, probabilidades e Softmax em LLMs](#)
- [Transformers: como funciona o mecanismo de atenção](#)
- [Transformers em PLN — IA Expert Academy](#)
- [Funções de ativação — IA Expert Academy](#)
- [ReLU, Sigmoid e Tanh — conceitos.tech](#)
- [Máquina de Vetores de Suporte — Wikipédia](#)
- [Classificação com SVM — Análise Macro](#)
- [Rede bayesiana — Wikipédia](#)
- [Comparando Classificadores: Árvores de Decisão, K-NN e Naive Bayes](#)
- [Lógica Fuzzy — UEM](#)
- [Lógica difusa — Wikipédia](#)
- [Sistema Especialista em IA — conceitos.tech](#)
- [Motor de inferência — Wikipédia](#)